

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 黄奕皓 学号 2024K8009929012 专业 计算机科学与技术
实验项目编号 4 实验名称 高速缓存 (Cache) 设计

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 /home/serve-ide/cod-lab/reports 目录下(注意: reports 全部小写)。文件命名规则: prjN.pdf, 其中 prj 和后缀名 pdf 为小写, N 为 1 至 4 的阿拉伯数字。例如: prj1.pdf。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: prj5-projectname.pdf, 其中“-”为英文标点符号的短横线。文件命名举例: prj5-dma.pdf。具体要求详见实验项目 5 讲义。
- 注 2: 使用 git add 及 git commit 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 git push 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

一、 逻辑电路结构与关键代码说明

本次实验基于哈佛结构设计实现了独立的指令缓存(I-Cache)与数据缓存(D-Cache), 二者均为 4 路组相联架构, 单块大小 32 字节, 总容量 1KB。I-Cache 采用轮替替换策略, 支持突发式缺失重填; D-Cache 采用写回 + 写分配策略, 支持脏块写回与不可缓存区域旁路访问。整体设计采用阻塞式工作模式, 当前请求处理完成前不接收新的访存请求, 最终通过标准基准程序完成了功能验证与性能评估。两款 Cache 的核心参数对比如下表所示:

表 1: I-Cache 与 D-Cache 核心参数对比

特性	I-Cache	D-Cache
总容量	1KB	1KB
组织方式	4 路组相联	4 路组相联
Cache 块大小	32 字节	32 字节
写策略	无(只读)	写回 (Write-back)+ 写分配 (Write-allocate)
替换策略	轮替替换 (Round-Robin)	轮替替换 (Round-Robin)
脏位存储	无	每块 1 位
旁路功能	无	支持不可缓存区域旁路
接口握手	Valid-Ready	Valid-Ready

32 位物理地址被统一划分为三个域: 24 位 Tag 域(地址 [31:8])、3 位 Index 域(地址 [7:5])、5 位 Offset 域(地址 [4:0]), 分别用于标签匹配、组索引选择与块内数据偏移。

1. 指令 Cache(I-Cache) 设计

- (a) 整体架构与接口指令 Cache 为纯只读缓存, 模块采用标准的 Valid-Ready 握手接口, CPU 侧支持 32 位指令读取, 内存侧支持 32 位突发传输, 一次缺失重填传输 8 拍数据。I-Cache 对外黑盒接口如下图所示:
- (b) 状态机整体设计 I-Cache 采用两段式有限状态机控制完整取指流程, 共包含 8 个状态, 覆盖命中读取、缺失替换、突发重填全路径。本设计为阻塞式 Cache, 仅在空闲状态接收 CPU 新请求, 当前请求未完成时不会响应新的访存, 保证处理逻辑的原子性。

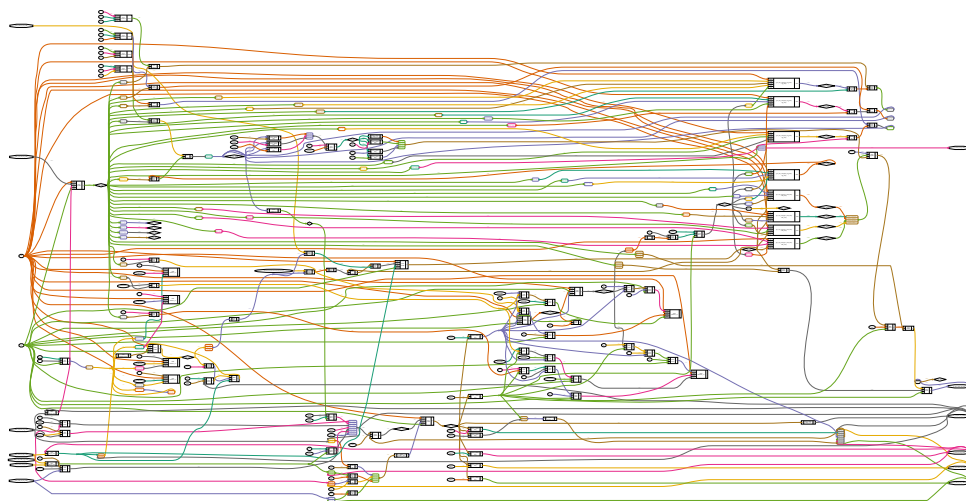


图 1: I-Cache 顶层模块黑盒接口图

核心工作流程分为两类:

- 命中流程: 空闲等待 → 读取 Tag 并比较 → 读取 Cache 数据 → 向 CPU 返回响应 → 回到空闲
- 缺失流程: 空闲等待 → 读取 Tag 并比较 → 选择替换路 → 发送内存读请求 → 接收突发数据 → 重填 Cache 行 → 向 CPU 返回响应 → 回到空闲

(c) 地址锁存与地址拆分接收 CPU 请求时锁存完整地址, 保证整个处理流程中地址稳定, 避免组合逻辑毛刺。同时将地址拆分为 Tag、Index、Offset 三个域, 分别用于标签比较、组选择和块内字选择。

Listing 1: 地址锁存与拆分逻辑

```
// 请求地址锁存与地址拆分
reg [31:0] req_addr;    // 锁存当前处理的请求地址
wire [23:0] req_tag;    // Tag域 [31:8]
wire [ 2:0] req_index;  // Index域 [7:5]
wire [ 4:0] req_offset; // Offset域 [4:0]
wire [ 2:0] word_sel;   // 32位字选择信号 [4:2]

assign req_tag = req_addr[31:8];
assign req_index = req_addr[7:5];
assign req_offset = req_addr[4:0];
assign word_sel = req_offset[4:2];

// 地址锁存: 保证处理过程中地址稳定
always @(posedge clk) begin
    if(rst) begin
        req_addr <= 32'd0;
    end
    else if(curr_state == S_WAIT && from_cpu_inst_req_valid) begin
        req_addr <= from_cpu_inst_req_addr;
    end
end
end
```

该设计的核心思想是: 仅在空闲状态接收新请求时更新地址寄存器, 其余状态保持地址不变, 确保 Tag 读取、数据读写等所有操作都基于同一个稳定地址。

- (d) 命中判断逻辑并行比较 4 路的有效位与 Tag 值,生成 4 位命中独热码,再转换为二进制路号。所有比较逻辑为纯组合逻辑,可在一个周期内完成命中判断。

Listing 2: 四路并行命中判断逻辑

```
// 命中判断信号
wire [3:0] hit_way_onehot; // 命中路独热码
wire      hit;           // 命中标志
reg [1:0] hit_way;       // 命中路编号 (二进制)

// 当前位置有效 + Tag匹配 = 该路命中
assign hit_way_onehot[0] = valid[0][req_index] & (tag_rdata[0] == req_tag);
assign hit_way_onehot[1] = valid[1][req_index] & (tag_rdata[1] == req_tag);
assign hit_way_onehot[2] = valid[2][req_index] & (tag_rdata[2] == req_tag);
assign hit_way_onehot[3] = valid[3][req_index] & (tag_rdata[3] == req_tag);
assign hit = |hit_way_onehot; // 四路按位或, 任意一路命中则整体命中

// 独热码转二进制路号
always @(*) begin
    case(hit_way_onehot)
        4'b0001: hit_way = 2'd0;
        4'b0010: hit_way = 2'd1;
        4'b0100: hit_way = 2'd2;
        4'b1000: hit_way = 2'd3;
        default: hit_way = 2'd0;
    endcase
end
```

- (e) 轮替替换控制通过计数器实现轮替替换指针,每次替换时将当前指针锁存到专用寄存器,同时指针自增指向下一路。所有替换操作均使用锁存后的路号,保证全流程路号稳定。

Listing 3: 轮替替换指针与路号锁存逻辑

```
// 替换控制
reg [1:0] evict_way_cnt; // 轮替替换指针
reg [1:0] evict_way_reg; // 当前流程锁定的替换路号

always @(posedge clk) begin
    if(rst) begin
        evict_way_cnt <= 2'd0;
        evict_way_reg <= 2'd0;
    end
    else if(curr_state == S_EVICT) begin
        evict_way_reg <= evict_way_cnt; // 锁存本次要替换的路
        evict_way_cnt <= evict_way_cnt + 2'd1; // 指针指向下一路
    end
end
```

- (f) Valid 位与阵列写控制 Valid 位用于标记 Cache 行是否存放了有效内容,复位时全部清零,替换时清除对应行的有效位,重填完成后置位。Tag 与 Data 阵列仅在重填状态下对替换路执行写操作,其余时

刻保持只读。

Listing 4: Valid 位与阵列写使能控制

```
// Valid位控制: 复位清0、替换清0、重填置1
always @(posedge clk) begin
    if(rst) begin
        valid[0] <= 8'd0;
        valid[1] <= 8'd0;
        valid[2] <= 8'd0;
        valid[3] <= 8'd0;
    end
    else if(curr_state == S_EVICT) begin
        valid[evict_way_reg][req_index] <= 1'b0;
    end
    else if(curr_state == S_REFILL) begin
        valid[evict_way_reg][req_index] <= 1'b1;
    end
end

// Tag/Data阵列写使能, 仅重填时对目标路写入
always @(*) begin
    tag_wen = 4'b0;
    data_wen = 4'b0;
    if(curr_state == S_REFILL) begin
        tag_wen[evict_way_reg] = 1'b1;
        data_wen[evict_way_reg] = 1'b1;
    end
end
```

- (g) 突发接收与重填逻辑接收内存突发返回的 8 拍 32 位数据, 按顺序拼接为 256 位完整 Cache 行, 重填时一次性写入对应路的 Data 阵列与 Tag 阵列。向内存发起的读请求地址会将原始地址低 5 位清零, 保证 32 字节块对齐, 对应一次完整的 Cache 行重填。

Listing 5: 突发数据接收与 Cache 重填逻辑

```
// 突发接收控制
reg [2:0] recv_cnt; // 接收数据拍数计数
reg [255:0] refill_data; // 重填数据缓存 (完整Cache行)

// 拼接8拍32位数据为256位Cache行
always @(posedge clk) begin
    if(rst) begin
        recv_cnt <= 3'd0;
        refill_data <= 256'd0;
    end
    else if(curr_state == S_RECV && from_mem_rd_rsp_valid) begin
        refill_data[{recv_cnt, 5'b0} +: 32] <= from_mem_rd_rsp_data;
        recv_cnt <= recv_cnt + 3'd1;
    end
    else if(curr_state == S_WAIT) begin
```

```

    recv_cnt <= 3'd0;
end
end

```

重填完成后采用快速返回策略: 直接从重填缓存中选择对应字返回 CPU, 不需要先写入 Cache 再读取, 减少一拍延迟。

- (h) 状态机核心转移采用两段式状态机设计, 时序逻辑寄存当前状态, 组合逻辑计算下一状态与输出。命中时走快速路径, 缺失时走替换-访存-重填长路径。

Listing 6: 核心状态转移逻辑

```

always @(*) begin
    next_state = curr_state;
    case(curr_state)
        S_WAIT: begin
            if(from_cpu_inst_req_valid) begin
                next_state = S_TAG_RD;
            end
        end
        S_TAG_RD: begin
            if(hit) next_state = S_CACHE_RD;
            else    next_state = S_EVICT;
        end
        S_EVICT: next_state = S_MEM_RD;
        S_MEM_RD: begin
            if(from_mem_rd_req_ready) begin
                next_state = S_RECV;
            end
        end
        S_RECV: begin
            if(from_mem_rd_rsp_valid && from_mem_rd_rsp_last) begin
                next_state = S_REFILL;
            end
        end
        S_REFILL: next_state = S_RESP;
        S_RESP: begin
            if(from_cpu_cache_rsp_ready) begin
                next_state = S_WAIT;
            end
        end
        default: next_state = S_WAIT;
    endcase
end

```

2. 数据 Cache(D-Cache) 设计

- (a) 整体架构与扩展特性数据 Cache 同样为 4 路组相联、1KB 容量、32 字节块大小，采用写回 (Write-back)+ 写分配 (Write-allocate) 策略。相比 I-Cache 增加了脏位存储、脏块写回、字节级写使能、不可缓存区域旁路四大核心特性。D-Cache 的完整整体架构如下页单独展示，包含 4 路存储阵列、脏位控制通路、写回重填通路与不可缓存旁路通路。

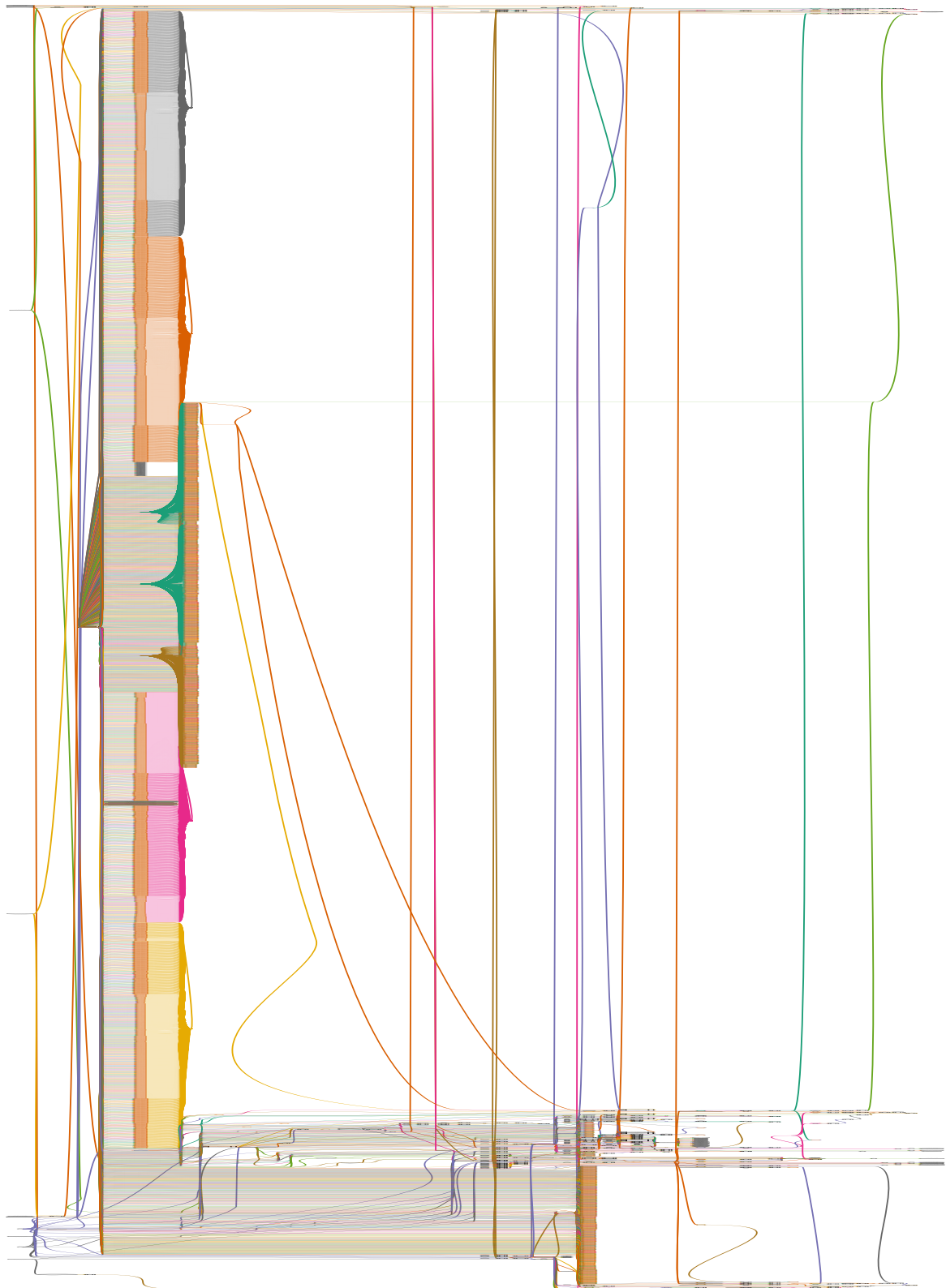


图 2: D-Cache 整体架构图: 包含 4 路存储阵列、脏位控制、旁路访问、写回重填全通路 (图实在太大了, 压缩了又太丑了, 实在没办法)

写策略的具体实现规则为：写命中时仅修改 **Cache** 数据并标记脏位，不立即写入内存，仅在替换脏块时才执行内存写回，大幅减少内存访问次数；写缺失时先将完整 **Cache** 行从内存读取重填，再合并写数据并标记脏位，而非直接写穿内存。

地址划分与 **I-Cache** 一致，同时定义两类不可缓存区域：**0x00000000 0x0000001F** 低地址区、**0x40000000** 及以上 **I/O** 空间，该类地址直接旁路到内存接口，不经过 **Cache** 存储。内存接口的突发长度 **len** 分两类场景：**Cache** 重填读与脏块写回时 **len=7**，对应 8 拍 32 位突发传输；不可缓存的旁路读写时 **len=0**，对应单拍 32 位传输。

- (b) 状态机设计 **D-Cache** 状态机在 **I-Cache** 基础上扩展为 15 个状态，核心增加了脏位检查状态、写回请求与数据状态、旁路读写状态。其中将原单周期替换拆分为路号锁存与脏位检查两个状态，解决时序不匹配问题。**D-Cache** 同样采用阻塞式工作模式，同一时刻仅处理一个访存请求。

核心工作流程分为四类：

- 读命中流程：空闲 → **Tag** 读取比较 → 命中读取数据 → 返回响应 → 空闲
- 读缺失 + 干净块替换：空闲 → **Tag** 比较 → 选择替换路 → 脏位检查 → 发内存读请求 → 接收重填 → 更新 **Cache** → 返回响应 → 空闲
- 写缺失 + 脏块写回：空闲 → **Tag** 比较 → 选择替换路 → 脏位检查 → 发写回请求 → 发送写回数据 → 发重填读请求 → 接收数据 → 合并写数据 → 更新 **Cache** → 空闲
- 不可缓存旁路：空闲 → 地址判断 → 发内存读写请求 → 数据交互 → 结束/返回响应 → 空闲

- (c) 脏位与有效位时序控制有效位与脏位均为时序寄存器，按状态更新：替换检查状态清除有效位与脏位，重填完成状态置有效位并按请求类型置脏位，写命中状态置脏位。

Listing 7: Valid 位与 Dirty 位控制逻辑

```
always @(posedge clk) begin
    if(rst) begin
        valid[0] <= 8'd0; valid[1] <= 8'd0;
        valid[2] <= 8'd0; valid[3] <= 8'd0;
        dirty[0] <= 8'd0; dirty[1] <= 8'd0;
        dirty[2] <= 8'd0; dirty[3] <= 8'd0;
    end
    else begin
        // 替换时失效对应块，同时清脏位
        if(curr_state == S_EVICT_CHECK) begin
            valid[evict_way_reg][req_index] <= 1'b0;
            dirty[evict_way_reg][req_index] <= 1'b0;
        end
        // 重填完成：有效位置1，脏位按读写类型设置
        if(curr_state == S_REFILL_DONE) begin
            valid[evict_way_reg][req_index] <= 1'b1;
            dirty[evict_way_reg][req_index] <= req_is_write ? 1'b1 : 1'b0;
        end
        // 写命中：脏位置1
        if(curr_state == S_HIT_PROC && req_is_write && hit) begin
            dirty[hit_way][req_index] <= 1'b1;
        end
    end
end
```


- (d) 字节级数据合并函数自定义函数实现按字节使能的数据合并, 支持字节、半字、字等不同位宽的写操作, 将 32 位写数据按掩码合并到 256 位 Cache 行的对应位置。

Listing 8: 字节使能数据合并函数

```
// 按字节使能将32位写数据合并到256位Cache行的指定字
function [255:0] merge_dcache_word;
    input [255:0] old_line;
    input [2:0] word_idx;
    input [31:0] wdata;
    input [3:0] wstrb;
    integer i;
    begin
        merge_dcache_word = old_line;
        for(i = 0; i < 4; i = i + 1) begin
            if(wstrb[i]) begin
                merge_dcache_word[{word_idx, 5'b0} + {i, 3'b0} +: 8]
                    = wdata[{i, 3'b0} +: 8];
            end
        end
    end
endfunction
```

该函数在写命中和写缺失重填两种场景下复用: 写命中时合并到读出的旧 Cache 行, 写缺失时合并到刚从内存读取的新 Cache 行。

- (e) 阵列写控制逻辑重填时写入完整 Tag 与 Cache 行, 写命中时仅写入对应路的部分数据。写数据通过上述数据合并函数生成, 保证字节使能的正确性。

Listing 9: D-Cache 阵列写控制逻辑

```
always @(*) begin
    tag_wen = 4'b0;
    data_wen = 4'b0;
    data_wdata = 256'd0;

    // 重填: 写入Tag和完整Data行
    if(curr_state == S_REFILL_DONE) begin
        tag_wen[evict_way_reg] = 1'b1;
        data_wen[evict_way_reg] = 1'b1;
        if(req_is_write) begin
            data_wdata = merge_dcache_word(refill_data, word_sel, req_wdata, req_wstrb);
        end
    end
    else begin
        data_wdata = refill_data;
    end
end

// 写命中: 合并后写入对应路的Data行
if(curr_state == S_HIT_PROC && req_is_write && hit) begin
    data_wen[hit_way] = 1'b1;
end
```

```

        data_wdata = merge_dcache_word(data_rdata[hit_way], word_sel, req_wdata,
        req_wstrb);
    end
end

```

- (f) 脏块写回控制替换时若目标块为脏块, 先将完整 Cache 行通过 8 拍突发写回内存。写回地址由被替换块的 Tag 与 Index 拼接而成, 每拍输出一个 32 位数据, 最后一拍拉高 last 信号。

Listing 10: 脏块写回突发控制逻辑

```

// 写回控制
reg [2:0] wr_cnt;          // 写回数据拍数计数
wire [31:0] wb_data;      // 当前写回的32位数据

assign wb_data = data_rdata[evict_way_reg][{wr_cnt, 5'b0}+: 32];

always @(posedge clk) begin
    if(rst) begin
        wr_cnt <= 3'd0;
    end
    else if(curr_state == S_WRITEBACK_DATA && from_mem_wr_data_ready) begin
        wr_cnt <= wr_cnt + 3'd1;
    end
    else if(curr_state == S_IDLE) begin
        wr_cnt <= 3'd0;
    end
end
end

```

- (g) 不可缓存旁路逻辑在空闲状态接收请求时, 先判断地址是否属于不可缓存区域。若是则直接走旁路路径, 不经过 Cache 存储阵列, 单拍完成请求发送与数据交互。

Listing 11: 不可缓存区域判断与旁路跳转

```

S_IDLE: begin
    if(from_cpu_mem_req_valid) begin
        // 按地址判断是否可缓存
        if( (from_cpu_mem_req_addr < 32'h00000020)
        || (from_cpu_mem_req_addr >= 32'h40000000) ) begin
            // 不可缓存, 走旁路
            if(from_cpu_mem_req) next_state = S_BYPASS_WR_REQ;
            else
                next_state = S_BYPASS_RD_REQ;
        end
        else begin
            // 可缓存, 进入Tag比较
            next_state = S_TAG_RD;
        end
    end
end
end

```

- (h) 拆分状态的时序优化设计将原单周期替换拆分为 S_EVICT 和 S_EVICT_CHECK 两个状态, 前者仅

负责锁存路号、更新轮替指针，后者使用稳定后的路号进行脏位判断与位清除。该设计匹配了时序逻辑的一拍延迟，从根本上解决了路号与脏位读取不同步的问题。

Listing 12: 拆分后的替换状态转移

```
S_EVICT: begin
    // 本拍只做路号锁存，下一拍再检查脏位
    next_state = S_EVICT_CHECK;
end

S_EVICT_CHECK: begin
    // 此时 evict_way_reg 已经稳定，再判断是否脏块
    if(dirty[evict_way_reg][req_index]) begin
        next_state = S_WRITEBACK_REQ;
    end
    else begin
        next_state = S_REFILL_REQ;
    end
end
end
```

3. 存储阵列模块说明 Tag 阵列与 Data 阵列为实验提供的基础同步存储模块，分别存储 24 位 Tag 标签与 256 位 Cache 行数据，均为单时钟一读一写端口。本次实验仅对数组定义表达式补充括号以保证不同工具下行为一致，未修改其核心功能。

二、实验过程中遇到的问题、思考过程及解决方法

1. I-Cache 替换路信号的时序不匹配问题

- 问题现象: I-Cache 缺失替换时偶发有效位清除错误，导致 Cache 行状态异常，出现非预期的命中或缺失，仿真中偶发指令数据错误。
- 思考过程: 最初替换路号直接通过组合逻辑从轮替计数器输出，没有寄存器锁存。轮替计数器是时序逻辑更新，组合逻辑输出存在毛刺与路径延迟，同一周期内用于清除有效位时，信号尚未稳定就被寄存器采样，导致写入错误路号。
- 解决方法: 增加 evict_way_reg 寄存器，在 S_EVICT 状态将当前轮替指针锁存，后续所有替换操作均使用该寄存器输出，同时轮替计数器在锁存后再更新，保证替换路号全流程稳定。

2. D-Cache 单周期替换的时序冲突问题

- 问题现象: D-Cache 替换脏块时，写回地址与数据错误，有时会将非脏块误判为脏块并执行写回，导致内存数据被错误覆盖。
- 思考过程: 最初将路号锁存、脏位判断、有效位清除都放在同一个 S_EVICT 状态完成。但替换路号寄存器是时序更新，必须等下一拍才稳定，单周期内直接用该寄存器读脏位，实际使用的是上一次替换的路号，逻辑完全错位。
- 解决方法: 将单个 S_EVICT 状态拆分为 S_EVICT 和 S_EVICT_CHECK 两个状态，前者仅锁存路号、更新计数器，后者再用稳定路号做脏位判断与位清除，通过拆分状态匹配时序延迟，解决了路号与脏位读取不同步的问题。

3. 存储阵列参数名修改失误问题

- 问题现象: 修改完 Cache 顶层逻辑后, 仿真出现大量不定态, Tag 与数据读取完全异常, 无法进行正常命中判断。
- 思考过程: 逐一排查端口连接, 发现修改存储阵列数组定义括号时, 误将端口地址参数名改错, 导致顶层与存储阵列端口连接错位, 地址与数据信号没有正确接入存储阵列, 读出数据全部为不定态。
- 解决方法: 对照实验原始代码修正 tag_array 与 data_array 的端口定义, 恢复正确的端口名称与连接关系, 不定态问题随之解决。

三、性能测试与分析

本次实验基于 FPGA 原型验证平台, 对实现完整指令缓存与数据缓存的 RISC-V 32 位处理器进行标准性能基准测试, 采用 CoreMark 与 Dhrystone 两款经典测试程序获取量化性能指标, 验证 Cache 功能正确性的同时评估系统整体运算性能, 为后续微架构优化提供数据支撑。

1. 实验环境与测试配置本次测试的硬件平台、软件工具链与测试参数如下表所示:

表 2: 性能测试环境与配置	
项目	说明
处理器架构	RISC-V 32 位(RV32), 支持完整 I-Cache 与 D-Cache
提交版本	c547f1de (feat(cache): implement full D-Cache logic & fix cache access bugs)
目标平台	NFV3 FPGA 原型验证板 (Runner 标签 nfv3_for_cod_v8)
软件工具链	GCC 8.2.0
基准程序	CoreMark 1.0, Dhrystone 2.1
测试方式	每款基准程序独立运行 5 次, 取平均值统计性能
运行配置	CoreMark 单次迭代 5 次, Dhrystone 单次运行 100 轮次

测试流程通过脚本自动完成: 加载比特流配置 FPGA, 完成 DDR 存储器初始化与系统复位后启动基准程序, 全程自动采集运行时间与结果校验值, 保证测试的可复现性。

2. CoreMark 基准测试结果 CoreMark 是面向嵌入式处理器的标准整型性能基准, 涵盖链表操作、矩阵运算、状态机等典型计算场景, 核心性能指标为每秒迭代次数 (Iterations per second)。本次测试 5 次运行的 CRC 校验值完全一致, 证明运算结果正确。原始运行时间数据如下:

表 3: CoreMark 测试原始数据			
运行编号	总耗时 (μ s)	迭代次数	正确性校验
1	5675223	5	通过
2	5675145	5	通过
3	5675250	5	通过
4	5675277	5	通过
5	5675264	5	通过

注: 测试日志中打印的 Iterations/Sec 值为 0 属于工程输出格式缺陷, 实际性能指标由总迭代次数与运行时间计算得到。

五次运行平均耗时为:

$$t_{avg} = \frac{5675223 + 5675145 + 5675250 + 5675277 + 5675264}{5} = 5675231.8 \mu\text{s} = 5.6752 \text{ s}$$

计算得到 CoreMark 性能分数:

$$\text{CoreMark 分数} = \frac{\text{总迭代次数}}{\text{总运行时间}} = \frac{5}{5.6752} \approx 0.88 \text{ Iterations/sec}$$

五次运行时间波动小于 150 微秒, 相对偏差不足 0.003%, 说明系统运行稳定, 测试环境干扰极小。该分值处于教学级简单处理器的典型水平, 反映出当前处理器单发射顺序架构、有限流水线深度与存储子系统延迟对性能的制约。

3. **Dhrystone** 基准测试结果 **Dhrystone** 是经典的整型运算性能基准, 主要测试处理器的通用指令执行效率, 常用指标为每秒 **Dhrystone** 次数与 **DMIPS** (1 **DMIPS** = 1757 **Dhrystones/sec**)。本次测试 5 次运行的变量最终值均与参考值完全匹配, 功能正确性验证通过。原始测试数据如下:

表 4: **Dhrystone** 测试原始数据

运行编号	总耗时 (μ s)	每秒 Dhrystone 次数	正确性校验
1	16465	6073	通过
2	16461	6074	通过
3	16462	6074	通过
4	16460	6075	通过
5	16461	6074	通过

五次运行平均吞吐量为 6074.0 **Dhrystones/sec**, 转换为 **DMIPS** 指标:

$$\text{DMIPS} = \frac{6074.0}{1757} \approx 3.46 \text{ DMIPS}$$

Dhrystone 测试结果与 **CoreMark** 呈现一致的性能量级, 多次测试偏差小于 2 **Dhrystones/sec**, 性能稳定性良好。由于 **Dhrystone** 程序代码量小、数据访问局部性强, 其表现优于 **CoreMark**, 也间接反映出 **CoreMark** 中更大规模的存储访问对 **Cache** 子系统的压力更为显著。

4. 功能正确性验证两款基准程序均通过了完整的结果校验:

- **CoreMark** 五次运行的 CRC 校验值 (seedcrc、crcfinal 等) 与标准期望值完全一致, 输出提示 'Correct operation validated'。
- **Dhrystone** 五次运行的所有全局与局部变量最终值均与参考值匹配, 无计算错误。
- 两次测试作业均以 'Hit good trap' 正常结束, 程序执行流程完整无误。

验证结果表明: 本次实现的指令缓存与数据缓存逻辑功能正确, 处理器能够稳定执行 C 语言编译的标准基准程序, 未出现指令读取错误、数据读写异常等 **Cache** 相关功能问题。

5. 性能分析与优化建议综合两款基准测试结果, 当前处理器在功能正确的基础上, 绝对性能处于较低水平, 主要瓶颈推测包括: 处理器工作主频较低、单发射顺序流水线指令级并行度有限、**Cache** 容量较小导致命中率不足、存储访问延迟较高。

针对后续优化方向, 提出以下建议:

- 确认并记录 **FPGA** 实际运行时钟频率, 计算 **CoreMark/MHz** 与 **DMIPS/MHz** 归一化指标, 便于不同架构横向对比。
- 添加硬件性能计数器, 统计 **Cache** 命中率、流水线停顿周期等微架构事件, 精准定位性能瓶颈。
- 与未开启 **Cache** 的基线处理器版本进行对比测试, 量化 **I-Cache** 与 **D-Cache** 引入带来的性能收益。
- 从微架构层面优化, 例如增加流水线深度、实现分支预测器、提升 **Cache** 容量与相联度, 进一步提升指令吞吐率。

四、 对讲义中思考题的理解和回答

1. 思考题 1: 如果用突发传输接口只传输 32-bit 数据($len=0$), **last** 信号在哪个位置拉高? 当 $len=0$ 时, 表示本次突发传输长度为 1 拍, 即仅传输 1 个 32 位数据。因此 **last** 信号会在第一个(也是唯一一个)数据节拍中与 **valid** 信号同时拉高, 标志当前数据就是本次突发的最后一拍。整个传输仅包含 1 次 **Valid-Ready** 握手, 即可完成单 32 位数据交互。
2. 思考题 2: 如何实现大于 4 字节的指令/数据交互? 基于 **Valid-Ready** 握手协议的突发 (**Burst**) 传输机制可以实现大于 4 字节的批量数据交互。具体实现为: 发起内存读写请求时, 通过 **len** 字段指定突发总拍数; 传输中每一拍数据都进行一次 **Valid-Ready** 握手, 通过 **last** 信号标记最后一拍; **Cache** 侧将多拍 32 位数据按顺序拼接为完整 **Cache** 行, 再写入存储阵列。本设计中 32 字节 **Cache** 行重填, 就是通过 $len=7$ 的 8 拍突发传输完成的。
3. 思考题 3: **CPU** 能否在下一个请求发送前一直保持接口信号, 替代 **Cache** 内部的地址锁存? 从功能上可以实现, 但不建议采用该设计。一方面 **CPU** 流水线存在停顿、跳转等复杂场景, 保持接口信号会增加 **CPU** 控制逻辑复杂度; 另一方面 **Cache** 作为独立模块, 内部锁存请求信号可以保证自身时序独立性, 降低 **CPU** 与 **Cache** 的耦合度, 便于模块复用与调试。因此标准设计中均由 **Cache** 侧完成请求信号锁存。

五、 实验所耗时间

在课后, 你花费了大约 10 小时完成此次实验。